Introdução à Orientação a Objetos

Aula 06 — Pacotes, constantes e membros estáticos

Programação Orientada a Objetos

Semana 6

Sumário

1. Introdução
2. Pacotes
3. Organizando o projeto em pacotes
4. Modificadores de acesso
5. Constantes
6. Membros estáticos
7. Exercício de fixação
8. Encerramento

Revisão da aula anterior

- Array é um **objeto**: precisa de `new` e de um tamanho fixo
- Índices vão de `0` até `length - 1`
- `length` é atributo do array; `length()` é método da `String`
- *enhanced-for* percorre sem índice e sem risco de estourar

Nosso projeto já tem `Livro`, `Autor`, `Produto`, `CadastroDeLivros` e `CarrinhoDeCompras`.

Vamos criar uma classe Date

A livraria precisa registrar a data de publicação. Criamos nossa própria classe:

```
1 public class Date {
2     private int dia;
3     private int mes;
4     private int ano;
5     // getters e setters
6 }
```

```
1 public static void main(String[] args) {
2     Date date = new Date();
3 }
```

Compila, executa, tudo certo.

Até que...

```java
import java.util.Date;

public class CadastroDeLivros {
    public static void main(String[] args) {
        Date date = new Date();  // qual Date?
    }
}
```

- Na API do Java **já existe** uma classe chamada `Date`
- Como a JVM sabe qual das duas você quer instanciar?

Resposta: em Java, classes são agrupadas em **pacotes**.

E tem um segundo problema

- Nossas classes estão todas soltas dentro da pasta `src`
- Só a aula passada acrescentou o `CarrinhoDeCompras` ao monte
- Com bibliotecas de terceiros e classes da API, o risco de nomes repetidos só cresce

Organização e ambiguidade de nomes: **o mesmo recurso resolve os dois**.

Hoje

- **Pacotes**: `package`, `import` e estrutura de pastas
- Os quatro **modificadores de acesso** — fechando a pendência da aula 03
- **Constantes** com `final` e `static final`
- **Membros estáticos**: o que pertence à classe, não ao objeto

O nome completo de uma classe

Para tirar a ambiguidade, podemos instanciar a classe pelo seu **nome completo**:

```
1 public static void main(String[] args) {
2     java.util.Date date = new java.util.Date();
3 }
```

fully qualified name = **nome do pacote** . **nome da classe**

O pacote da classe `Date` da API é `java.util`.

Legibilidade: o `import`

Escrever o nome completo em toda linha é ilegível:

```
1 java.util.Date date = new java.util.Date();
```

O `import` declara o nome completo **uma vez**, no início do arquivo:

```
1 import java.util.Date;
2
3 class Teste {
4     public static void main(String[] args) {
5         Date date = new Date();
6     }
7 }
```

Quando o nome completo ainda é necessário

- Só quando você precisa usar **duas classes de mesmo nome** no mesmo arquivo
- Nesse caso: `import` em uma delas, nome completo na outra

`import` não muda o que o programa faz — muda o que você precisa digitar.

O pacote que você nunca importou

```java
String nome = "Java 8 Pratico";
System.out.println(nome);
double raiz = Math.sqrt(16);
```

- `String`, `System` e `Math` estão no pacote `java.lang`
- `java.lang` é o **pacote padrão** do Java: importado automaticamente
- Por isso você programa desde a aula 01 sem nunca ter escrito um `import`

E as nossas classes, em que pacote estão?

- Criamos tudo direto na pasta `src`, sem declarar pacote nenhum
- Elas estão no *default package* — o "pacote nenhum"

Isso não é nada bom.

Toda classe deve ser agrupada em um pacote.

Nomenclatura padrão dos pacotes

Por convenção, um pacote em Java sempre:

- é escrito em letra minúscula (*lowercase*);
- usa um nome de **domínio invertido**, iniciado por `com`, `edu`, `gov` etc.

Domínio	**Pacote**
`casadocodigo.com.br`	`br.com.casadocodigo`
`alura.com.br`	`br.com.alura`
`ifsp.edu.br`	`br.edu.ifsp`

Convenções oficiais: `oracle.com/technetwork/java/codeconventions-135099.html`

Os pacotes da nossa livraria

Pacote	Classes
`br.edu.ifsp.livraria`	`Autor`, `CarrinhoDeCompras`
`br.edu.ifsp.livraria.produtos`	`Livro`, `Produto`
`br.edu.ifsp.livraria.testes`	`CadastroDeLivros`

- Um pacote agrupa classes que tratam do **mesmo assunto**
- As classes com `main` ficam separadas — em um projeto real não haveria várias

Movendo as classes

Na IDE: *Refactor > Move...* (ou arrastando com o mouse).

- Ao mover, vários **erros de compilação** vão aparecer
- **Termine de mover todas as classes antes de corrigir qualquer erro**
- Corrigir no meio do caminho só gera retrabalho

De novo: o erro de compilação é o compilador te mostrando o trabalho que falta.

A ordem das instruções no arquivo

```java
package br.edu.ifsp.livraria.testes;

import br.edu.ifsp.livraria.Autor;
import br.edu.ifsp.livraria.produtos.Livro;

public class CadastroDeLivros {
    public static void main(String[] args) {
        Autor autor = new Autor();
        autor.setNome("Rodrigo Turini");
    }
}
```

package $\rightarrow$ import(s) $\rightarrow$ declaração da classe

Duas regras para atravessar pacotes

1. Para uma classe ser visível em **outro pacote**, ela precisa ser `public`
2. Toda classe `public` deve estar em um arquivo `.java` com o **mesmo nome** dela

- A classe `Livro` mora obrigatoriamente em `Livro.java`
- Isso já era boa prática desde a aula 01 — agora virou obrigação
- O `import` é sempre necessário para usar classes de outros pacotes

Importando o pacote inteiro

Em vez de:

```
1 import br.edu.ifsp.livraria.Autor;
2 import br.edu.ifsp.livraria.Editora;
```

é possível escrever:

```
1 import br.edu.ifsp.livraria.*;
```

- **Mito:** isso deixa o programa mais lento — não afeta o tempo de execução
- **Problema real:** não previne a ambiguidade de nomes
- Recomendação: importe **classe a classe**

Pacote é pasta

`br.edu.ifsp.livraria.produtos.Livro`

$\Downarrow$

`src/br/edu/ifsp/livraria/produtos/Livro.java`

- Cada ponto do nome do pacote é uma pasta no sistema de arquivos
- Não é metáfora: abra o explorador de arquivos e confira

Ainda falta um erro

Depois de mover tudo, o `CadastroDeLivros` continua sem compilar:

```
The method mostrarDetalhes() from the type Livro is not visible
```

- O método estava sem modificador de acesso
- Isso bastava enquanto todas as classes estavam no mesmo lugar
- Assim como as classes, um **método** precisa ser `public` para ser visto de outro pacote

```
public void mostrarDetalhes() {
    System.out.println("Nome: " + nome);
}
```

Os quatro modificadores de acesso

- `public` — acessível por qualquer classe do projeto
- `protected` — própria classe, mesmo pacote e classes filhas
- *default* (nenhum modificador escrito) — somente o **mesmo pacote**
- `private` — somente a própria classe

Valem para atributos, métodos e construtores.

Tabela de visibilidade

	`private`	*default*	`protected`	`public`
Mesma classe	sim	sim	sim	sim
Mesmo pacote	não	sim	sim	sim
Outro pacote, classe filha	não	não	sim	sim
Outro pacote, qualquer classe	não	não	não	sim

- A linha "classe filha" só faz sentido completo na **aula 10** (herança)
- Por enquanto, leia `protected` como "*default* + filhas"

O modificador *default*

```java
package br.edu.ifsp.livraria.produtos;

class CalculadoraDeFrete {   // sem modificador
    double calcula(double peso) {
        return peso * 1.5;
    }
}
```

- Visível para as outras classes de `br.edu.ifsp.livraria.produtos`
- Invisível para `br.edu.ifsp.livraria.testes`
- Uso: detalhe interno do pacote, que não faz parte da "fachada" pública

Duas regras que valem para classes

- Uma classe de topo só admite **dois** níveis: *default* ou `public`
- Classes **não** podem ser `private` nem `protected`

Pense: se uma classe fosse `private`, quem conseguiria usá-la?

Atenção: organizar em pacotes não afrouxa o encapsulamento.
Atributo continua `private`, sempre.

Números mágicos espalhados pelo código

```java
public void aplicaDescontoDe(double porcentagem) {
    if (porcentagem > 0.3) {
        System.out.println("Desconto acima do permitido");
        return;
    }
    this.valor -= this.valor * porcentagem;
}
```

- O `0.3` aparece aqui, no `Produto`, no relatório, no cadastro...
- Se a política mudar para 50%, quantos arquivos você precisa abrir?
- E quem lê o código: `0.3` de quê?

A palavra reservada `final`

```
1 public class Livro {
2     private final double descontoMaximo = 0.3;
3 }
```

- Um atributo `final` recebe valor **uma vez** e não muda mais
- Tentar reatribuir gera erro de compilação

```
1 this.descontoMaximo = 0.5;  // nao compila
```

Mas ainda existe uma cópia desse valor **em cada livro criado** — e ele é o mesmo para todos.

static final: a constante da classe

```java
public class Livro {
    public static final double DESCONTO_MAXIMO = 0.3;

    public void aplicaDescontoDe(double porcentagem) {
        if (porcentagem > DESCONTO_MAXIMO) {
            System.out.println("Desconto acima do permitido");
            return;
        }
        this.valor -= this.valor * porcentagem;
    }
}
```

```java
System.out.println(Livro.DESCONTO_MAXIMO);  // sem criar objeto
```

Convenção de nomes para constantes

```
DESCONTO_MAXIMO,  ESTOQUE_MINIMO,  TAXA_DE_ENTREGA
```

- Tudo em **maiúsculas**, palavras separadas por _
- É convenção de estilo, não regra do compilador — mas todo mundo segue
- Ler `MAIUSCULA` no meio do código já avisa: "isso aqui não muda"

Constantes que você já usa

```
1 System.out.println(Math.PI);                // 3.141592653589793
2 System.out.println(Integer.MAX_VALUE);      // 2147483647
3 System.out.println(Double.MIN_VALUE);
```

- Nenhum `new` foi escrito: são acessadas pelo **nome da classe**
- São exatamente `public static final` da API do Java
- Repare que já seguem a convenção de maiúsculas

Quantos livros já foram cadastrados?

- Cada `Livro` tem seu nome, seu valor, seu autor
- Mas o **total de livros cadastrados** não pertence a nenhum livro específico
- Perguntar isso a um objeto `Livro` seria estranho: `livro.getTotal()`?

Existe informação que pertence à **classe**, não ao objeto.

Atributo de instância x atributo estático

```java
public class Livro {
    private String nome;                  // um por objeto
    private static int totalCadastrados; // um para todos
}
```

sem `static`	cada objeto tem a sua cópia
com `static`	existe **uma única** cópia, compartilhada

Quando um objeto muda um atributo estático, ele muda para **todos**.

Contando os livros no construtor

```java
public class Livro {
    private String nome;
    private static int totalCadastrados;

    public Livro(Autor autor) {
        this.autor = autor;
        totalCadastrados++;
    }
}
```

- O construtor roda uma vez por objeto — o contador é incrementado a cada `new`
- Repare: **sem** `this.` — o atributo não é do objeto

Método estático

```java
public static int getTotalCadastrados() {
    return totalCadastrados;
}
```

```java
new Livro(autor);
new Livro(autor);
new Livro(autor);

System.out.println(Livro.getTotalCadastrados());  // 3
```

- Chamado pelo **nome da classe**, sem nenhum objeto criado
- `Math.sqrt`, `Integer.parseInt`: você já usa métodos estáticos há semanas

Restrições de um método estático

```java
public class Livro {
    private String nome;
    private static int totalCadastrados;

    public static void mostraResumo() {
        System.out.println(totalCadastrados);  // ok
        System.out.println(this.nome);         // NAO COMPILA
    }
}
```

Um método `static`:

- só acessa diretamente atributos `static` da sua classe;
- só chama diretamente outros métodos `static`;
- **não tem** `this` — não existe "objeto atual".

Por que `main` é `static`?

```
1 public static void main(String[] args) {
2 }
```

- Quando o programa começa, **nenhum objeto existe ainda**
- A JVM precisa chamar o `main` sem ter nada para instanciar
- Só é possível chamar um método sem objeto se ele for `static`

E é por isso que dentro do `main` você sempre precisa criar objetos antes de usá-los.

Bloco estático *(leitura complementar)*

```java
public class TabelaDeFrete {
    static double taxaBase;

    static {
        taxaBase = Math.sqrt(2.0) * 10;
        System.out.println("Tabela carregada");
    }
}
```

- Executado **uma vez**, quando a classe é carregada
- Roda **antes** de qualquer construtor
- Serve para inicializações que não cabem em uma única linha

Conteúdo de leitura: não será cobrado na lista nem na prova.

Cuidado: `static` não é "variável global"

- Um atributo estático é compartilhado por **todo** o programa
- Usar `static` para "facilitar o acesso" destrói o encapsulamento
- Todo objeto passa a poder alterar o estado de todos os outros

Pergunta-teste:

esse dado é do objeto ou da classe?

Se for do objeto, não é `static`.

Organizando a classe Produto

Partindo da Produto construída nas aulas anteriores:

```
1 public class Produto {
2     private String nome;
3     private double preco;
4     private int quantidadeEmEstoque;
5
6     public Produto(String nome, double preco) { ... }
7     public boolean retiraDoEstoque(int quantidade) { ... }
8 }
```

1. Mova Produto para br.edu.ifsp.livraria.produtos e a classe de teste para br.edu.ifsp.livraria.testes; ajuste package, import e visibilidade até compilar
2. Crie as constantes ESTOQUE_MINIMO (5) e DESCONTO_MAXIMO (0.3)
3. Crie um contador estático de produtos cadastrados, com getTotalCadastrados()
4. Tente criar um método static que imprima o nome do produto. O que acontece?

Solução — pacotes

```java
package br.edu.ifsp.livraria.produtos;

public class Produto {
    public boolean retiraDoEstoque(int quantidade) { ... }
}
```

```java
package br.edu.ifsp.livraria.testes;

import br.edu.ifsp.livraria.produtos.Produto;

public class TesteProduto {
    public static void main(String[] args) { ... }
}
```

A classe **e** os métodos usados de fora precisaram virar `public`.

Solução — constantes e contador

```java
public class Produto {
    public static final int ESTOQUE_MINIMO = 5;
    public static final double DESCONTO_MAXIMO = 0.3;
    private static int totalCadastrados;
    private String nome;
    private double preco;

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
        totalCadastrados++;
    }

    public static int getTotalCadastrados() {
        return totalCadastrados;
    }
}
```

Solução — o item 4 era uma armadilha

```java
public static void mostraNome() {
    System.out.println(nome);  // erro de compilacao
}
```

```
non-static variable nome cannot be referenced from a static context
```

- `nome` pertence a **um** produto; o método estático não sabe a qual
- O nome de um produto é dado **do objeto** — o método tem que ser de instância
- Já o total de cadastrados é dado **da classe** — por isso ele pode ser estático

Resumo da aula

- Pacote resolve **organização** e **ambiguidade de nomes**
- `package` → `import` → classe; pacote é pasta no disco
- Para atravessar pacotes: classe e método precisam ser `public`
- Quatro modificadores: `private`, *default*, `protected`, `public`
- `static final` = constante da classe, em `MAIUSCULAS`
- `static` = pertence à classe, não ao objeto; não tem `this`
- `main` é `static` porque a JVM o chama sem objeto nenhum

Próxima aula

- **Revisão geral** e **Prova 1**
- A prova cobre tudo desde a aula 01: fundamentos, classes e objetos, encapsulamento, construtores, sobrecarga, arrays, pacotes e estáticos

Traga suas dúvidas da lista — a aula de revisão é para isso.

Tarefa: lista de exercícios da aula 06.